Employee Salary Analysis Program

```
================================================ REST
Total salary 605000
Average of the salary 121000.0
minimum salary is 90000
maximum of the salary 160000
```

File Edit Format Run Options Window Help

```python
a=(125000,130000,100000,160000,90000)
total=sum(a)
average=total/len(a)
min=min(a)
max=max(a)
print("Total salary",total,"\nAverage of the salary",average,"\nminimum salary i
```

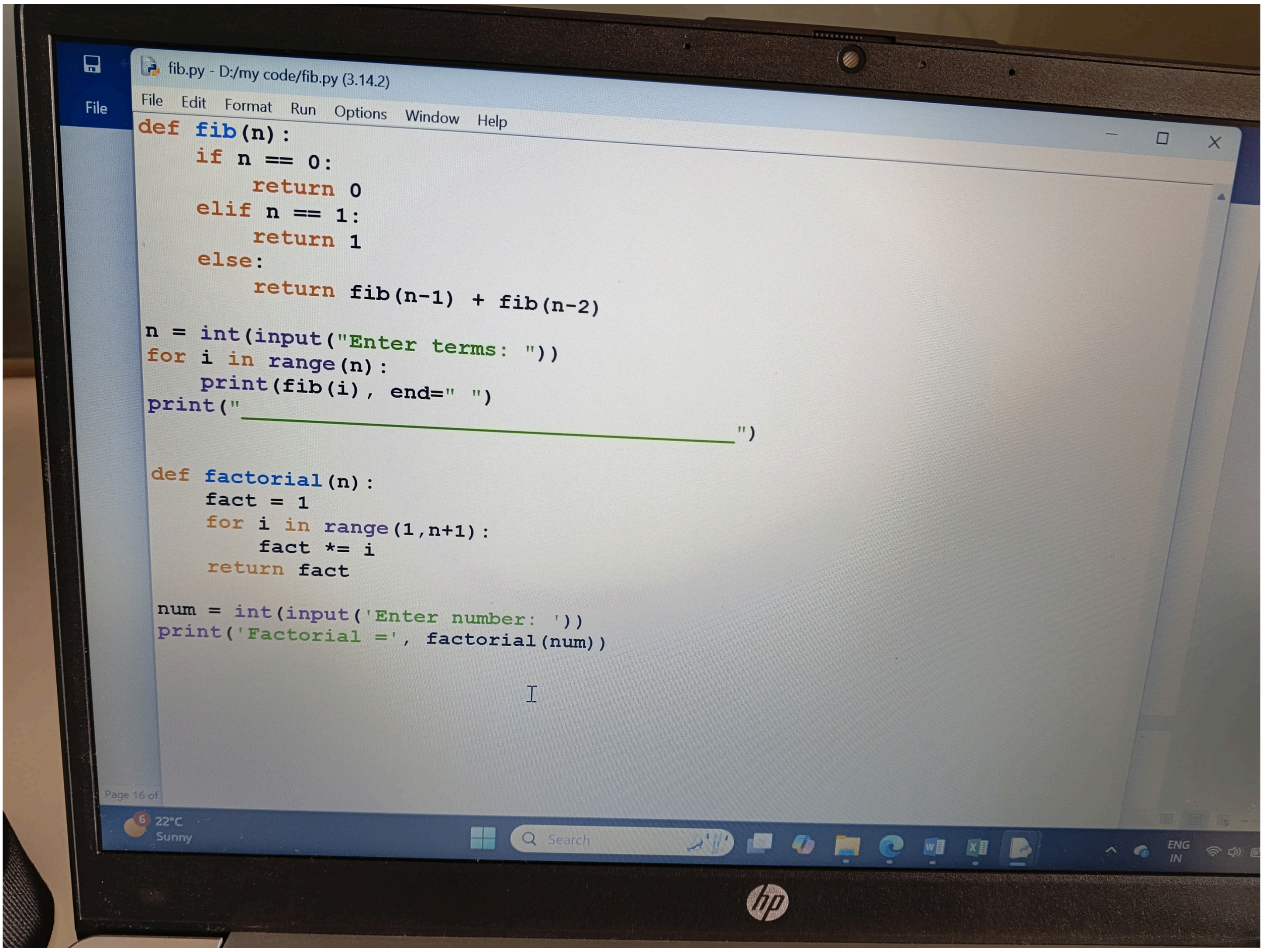
Fibonacci Series Using Recursion

```
Enter the number :5
0 1 1 2 3
```

fino.py - D:\new\30-01-2026\fino.py (3.11.9)

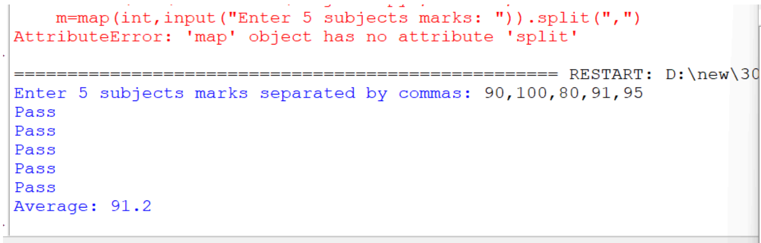
File Edit Format Run Options Window Help

```python
n=int(input("Enter the number :"))
def f(n):
    if n==0:
        return 0
    elif n==1:
        return 1
    else:
        return f(n-1) + f(n-2)
for i in range(n):
    print(f(i),end=" ")
```

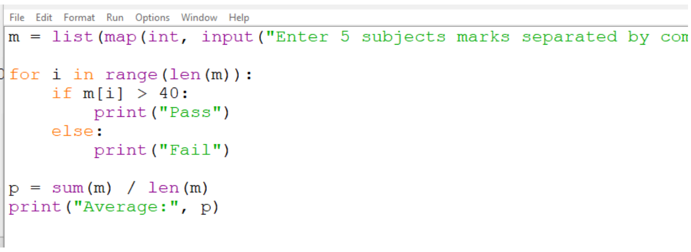


```
def fib(n):
    if n == 0:
        return 0
    elif n == 1:
        return 1
    else:
        return fib(n-1) + fib(n-2)

n = int(input("Enter terms: "))
for i in range(n):
    print(fib(i), end=" ")
print("____________________________________________________")


def factorial(n):
    fact = 1
    for i in range(1,n+1):
        fact *= i
    return fact

num = int(input('Enter number: '))
print('Factorial =', factorial(num))
```

Student Marks Pass/Fail & Average Program



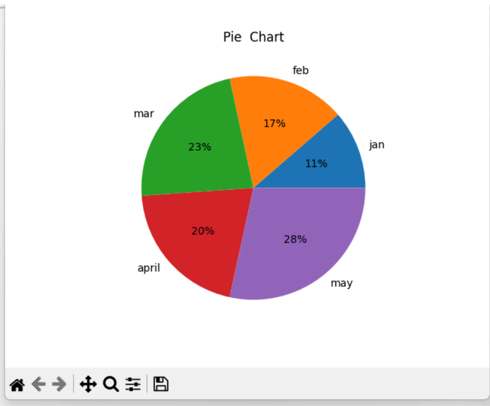
```
    m=map(int,input("Enter 5 subjects marks: ")).split(",")
AttributeError: 'map' object has no attribute 'split'

==================================================== RESTART: D:\new\30
Enter 5 subjects marks separated by commas: 90,100,80,91,95
Pass
Pass
Pass
Pass
Pass
Average: 91.2
```

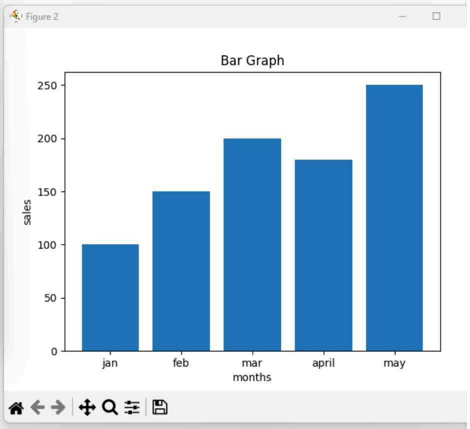


```
m = list(map(int, input("Enter 5 subjects marks separated by com
for i in range(len(m)):
    if m[i] > 40:
        print("Pass")
    else:
        print("Fail")

p = sum(m) / len(m)
print("Average:", p)
```

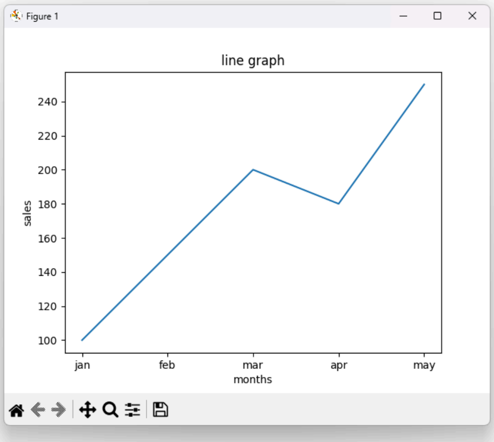
Python Matplotlib Multiple Graphs Program

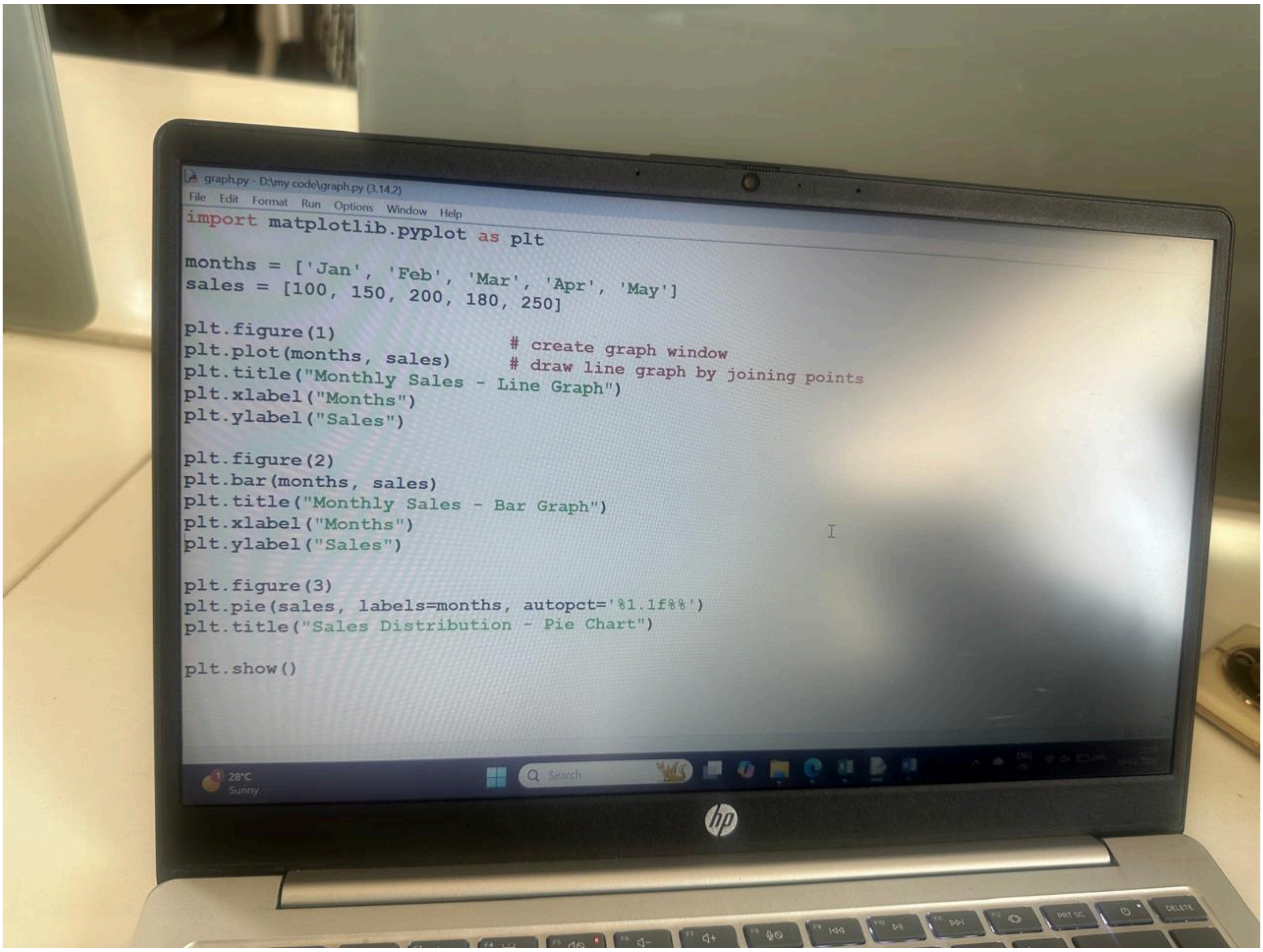
```python
import matplotlib.pyplot as plt

months = ['jan', 'feb', 'mar', 'april', 'may']
sales = [100, 150, 200, 180, 250]

plt.figure(1)
plt.plot(months, sales)
plt.title("Line Graph")
plt.xlabel("months")
plt.ylabel("sales")

plt.figure(2)
plt.bar(months, sales)
plt.title("Bar Graph")
plt.xlabel("months")
plt.ylabel("sales")

plt.figure(3)
plt.pie(sales, labels=months, autopct='%1.1f%%')
plt.title("Pie  Chart")

plt.show()
```

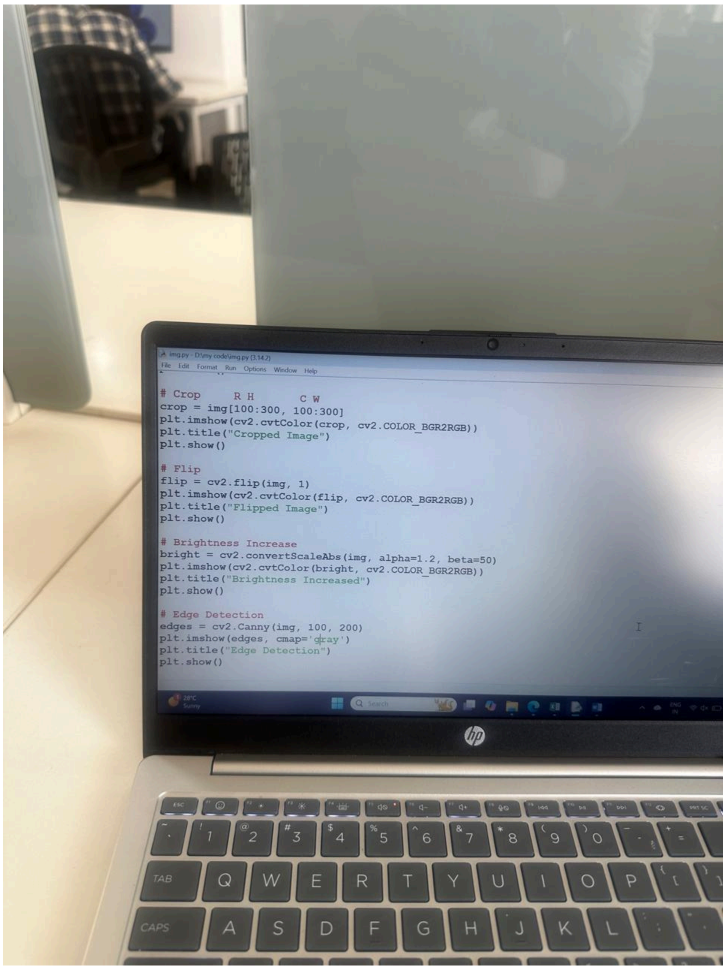




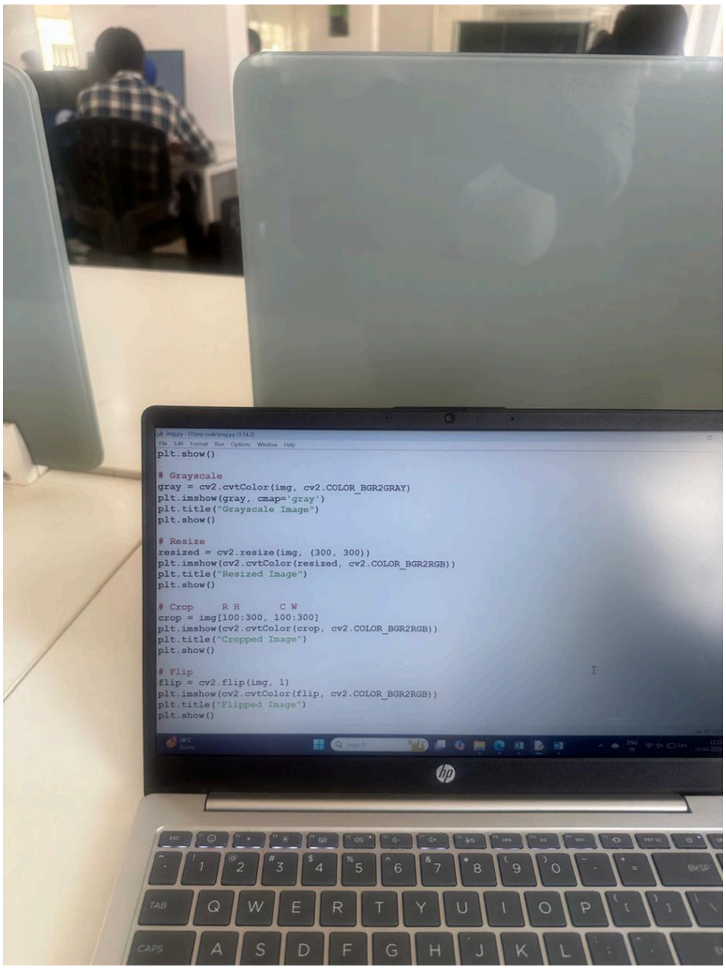

```python
import matplotlib.pyplot as plt

months=['jan', 'feb', 'mar', 'apr', 'may']
sales=[100,150,200,180,250]

students=[1,2,3,4,5]
marks=[60,75,80,70,90]

subjects = ['math', 'science', 'english']
marks_sub=[85,90,75]

plt.plot(months, sales)
plt.title("line graph")
plt.xlabel("months")
plt.ylabel("sales")
plt.show()

plt.bar(months, sales)
plt.title("Bar graph")
plt.xlabel("months")
plt.ylabel("sales")
plt.show()

plt.pie(sales, labels=months, autopct='%1.1f%%')
plt.title("pie graph")
plt.show()

plt.scatter(students, marks)
plt.title("scatter graph")
plt.xlabel("students")
plt.ylabel("marks")
plt.show()

plt.hist(marks)
plt.title("histogram")
plt.xlabel("Marks")
plt.ylebel("Frequency")
plt.show()

plt.barh(sunjects, marks_sub)
plt.title("horizontal bar graph")
plt.xlabel("Marks")
```



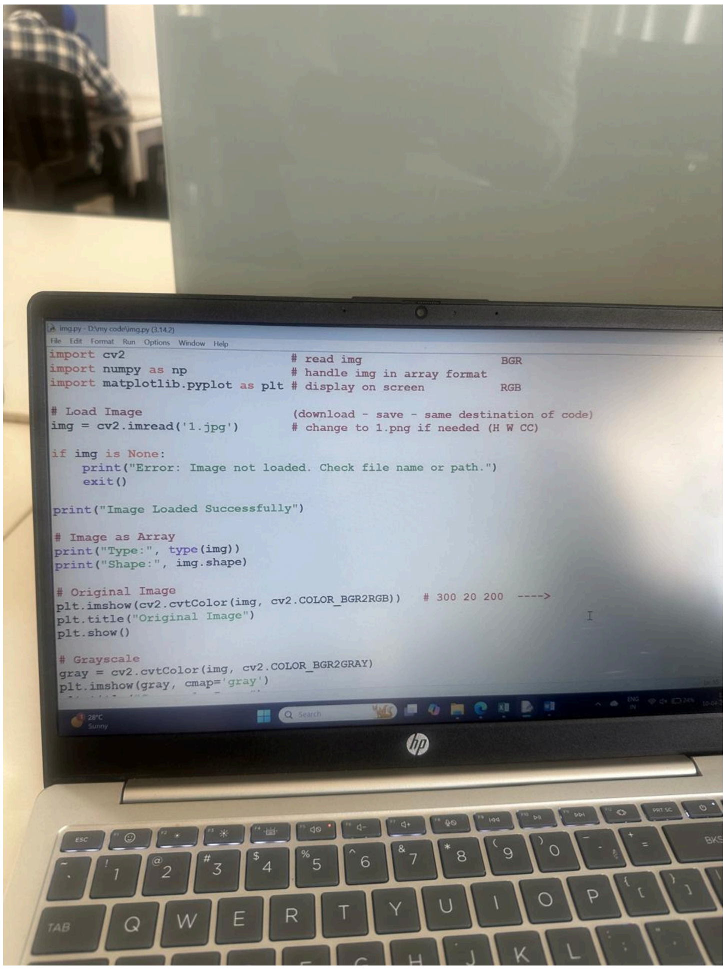

graph.py - D:\my code\graph.py (3.14.2)
File Edit Format Run Options Window Help
import matplotlib.pyplot as plt
months = ['Jan', 'Feb', 'Mar', 'Apr', 'May']
sales = [100, 150, 200, 180, 250]
plt.figure(1)   # create graph window
plt.plot(months, sales)   # draw line graph by joining points
plt.title("Monthly Sales - Line Graph")
plt.xlabel("Months")
plt.ylabel("Sales")
plt.figure(2)
plt.bar(months, sales)
plt.title("Monthly Sales - Bar Graph")
plt.xlabel("Months")
plt.ylabel("Sales")
plt.figure(3)
plt.pie(sales, labels=months, autopct='%1.1f%%')
plt.title("Sales Distribution - Pie Chart")
plt.show()
28°C
Sunny
Search
hp

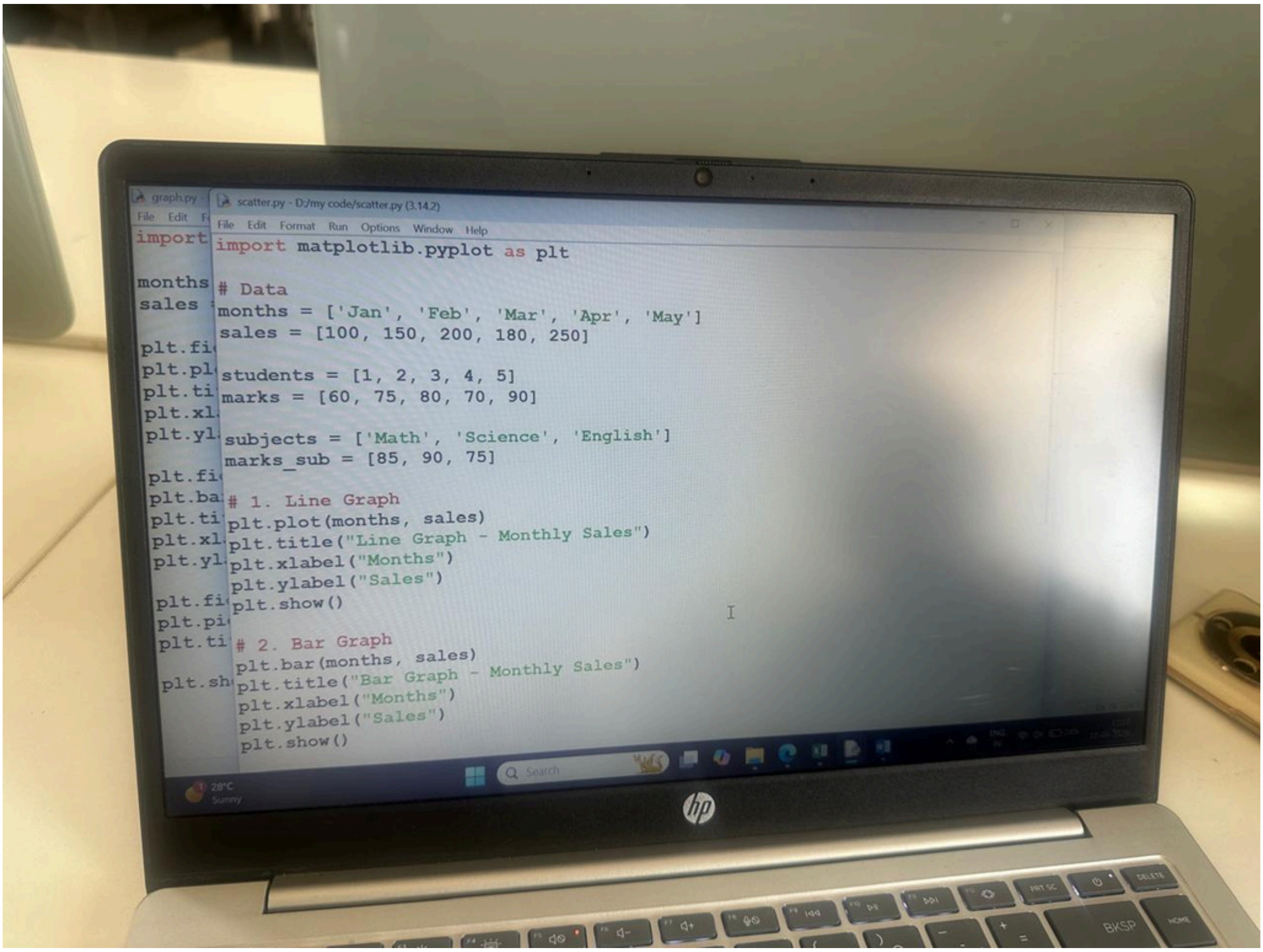


# Crop   R H   C W
crop = img[100:300, 100:300]
plt.imshow(cv2.cvtColor(crop, cv2.COLOR_BGR2RGB))
plt.title("Cropped Image")
plt.show()
# Flip
flip = cv2.flip(img, 1)
plt.imshow(cv2.cvtColor(flip, cv2.COLOR_BGR2RGB))
plt.title("Flipped Image")
plt.show()
# Brightness Increase
bright = cv2.convertScaleAbs(img, alpha=1.2, beta=50)
plt.imshow(cv2.cvtColor(bright, cv2.COLOR_BGR2RGB))
plt.title("Brightness Increased")
plt.show()
# Edge Detection
edges = cv2.Canny(img, 100, 200)
plt.imshow(edges, cmap='gray')
plt.title("Edge Detection")
plt.show()



plt.show()
# Grayscale
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
plt.imshow(gray, cmap='gray')
plt.title("Grayscale Image")
plt.show()
# Resize
resized = cv2.resize(img, (300, 300))
plt.imshow(cv2.cvtColor(resized, cv2.COLOR_BGR2RGB))
plt.title("Resized Image")
plt.show()
# Crop   R H   C W
crop = img[100:300, 100:300]
plt.imshow(cv2.cvtColor(crop, cv2.COLOR_BGR2RGB))
plt.title("Cropped Image")
plt.show()
# Flip
flip = cv2.flip(img, 1)
plt.imshow(cv2.cvtColor(flip, cv2.COLOR_BGR2RGB))
plt.title("Flipped Image")
plt.show()


img.py - D:\my code\img.py (3.14.2)
File Edit Format Run Options Window Help
import cv2 # read img BGR
import numpy as np # handle img in array format
import matplotlib.pyplot as plt # display on screen RGB
# Load Image (download - save - same destination of code)
img = cv2.imread('1.jpg') # change to 1.png if needed (H W CC)
if img is None:
print("Error: Image not loaded. Check file name or path.")
exit()
print("Image Loaded Successfully")
# Image as Array
print("Type:", type(img))
print("Shape:", img.shape)
# Original Image
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB)) # 300 20 200 ---->
plt.title("Original Image")
plt.show()
# Grayscale
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
plt.imshow(gray, cmap='gray')
28°C
Sunny
Search

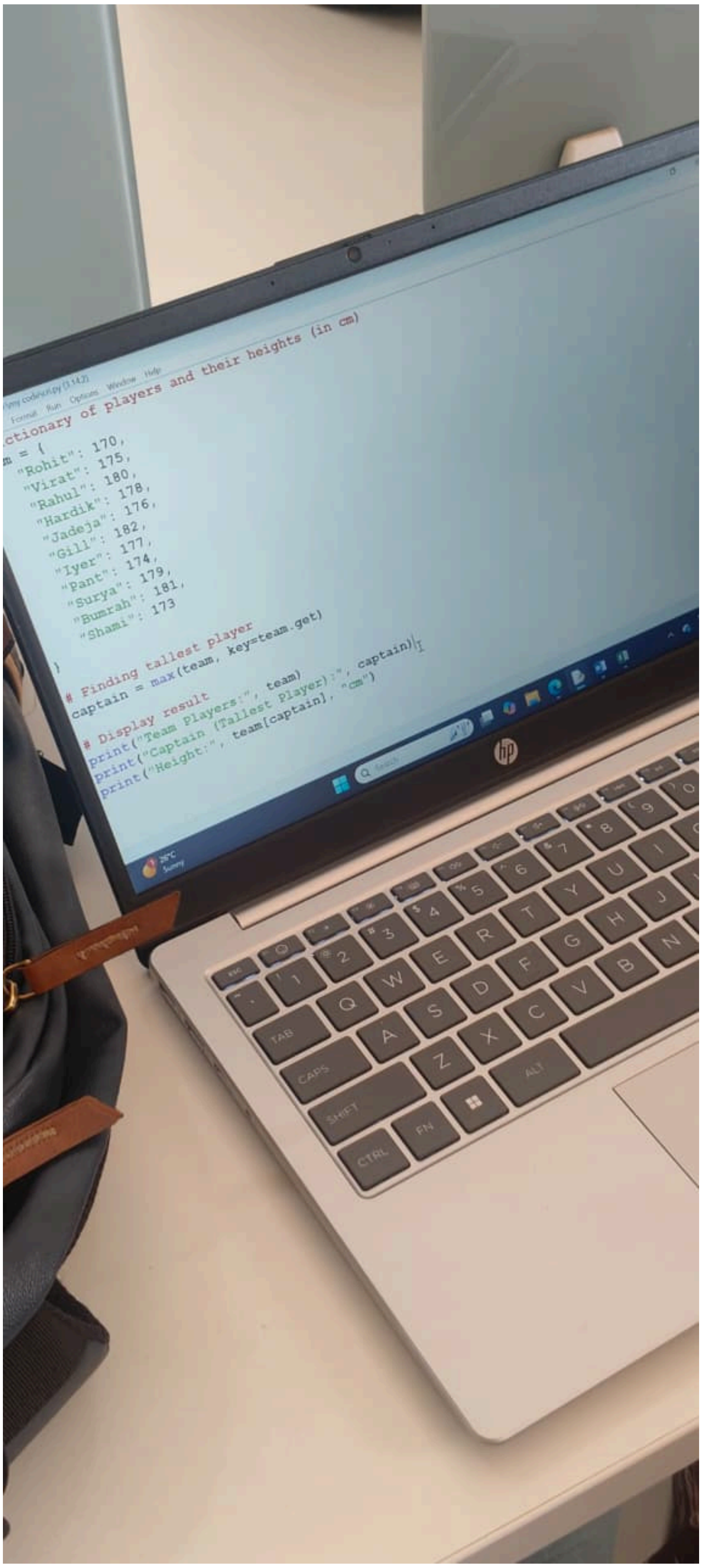

scatter.py - D:/my code/scatter.py (3.14.2)
File Edit Options Run Format Window Help
import matplotlib.pyplot as plt
# Data
months = ['Jan', 'Feb', 'Mar', 'Apr', 'May']
sales = [100, 150, 200, 180, 250]
students = [1, 2, 3, 4, 5]
marks = [60, 75, 80, 70, 90]
subjects = ['Math', 'Science', 'English']
marks_sub = [85, 90, 75]
# 1. Line Graph
plt.plot(months, sales)
plt.title("Line Graph - Monthly Sales")
plt.xlabel("Months")
plt.ylabel("Sales")
plt.show()
# 2. Bar Graph
plt.bar(months, sales)
plt.title("Bar Graph - Monthly Sales")
plt.xlabel("Months")
plt.ylabel("Sales")
plt.show()
28°C
Sunny
Search

Leap Year Checking Program

```
n=int(input("Enter the year"))
if(n%100==0):
    if n%400==0:
        print("l.y")
    else:
        print("not l.y")
else:
    if n%4==0:
        print("l.y")
    else:
        print("not ly")
```

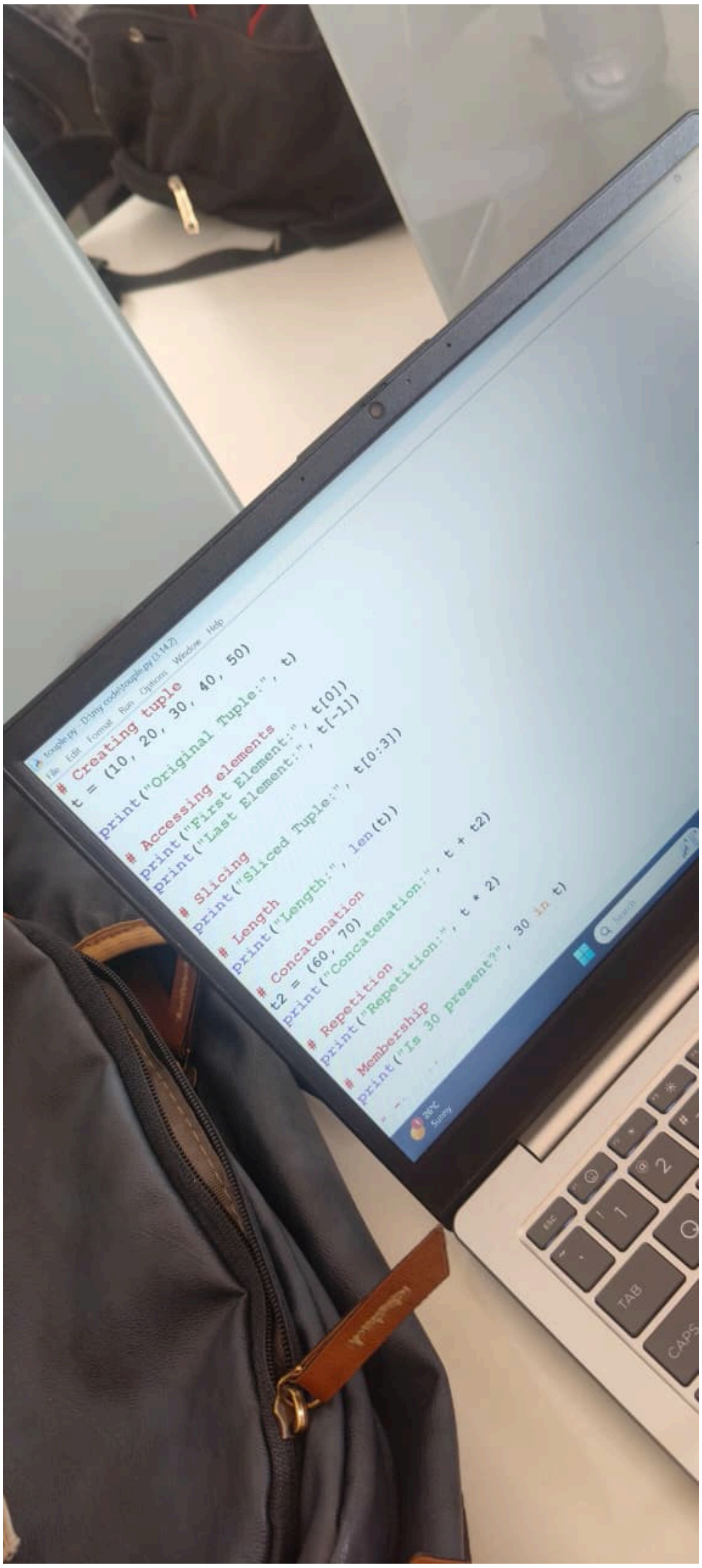
```
    ======================== RESTART: D:\
>>>
    ======================== RESTART: D:\
>>>
    ==================== RESTART: D:\new\
    Enter the year2025
    not ly
>>>
```

Dictionary Program to Find Tallest Player



ctionary of players and their heights (in cm)
m = {
"Rohit": 170,
"Virat": 175,
"Rahul": 180,
"Hardik": 178,
"Jadeja": 176,
"Gill": 182,
"Iyer": 177,
"Pant": 174,
"Surya": 179,
"Bumrah": 181,
"Shami": 173
}
# Finding tallest player
captain = max(team, key=team.get)
# Display result
print("Team Players:", team)
print("Captain (Tallest Player):", captain)
print("Height:", team[captain], "cm")
hp

Tuple Operations in Python

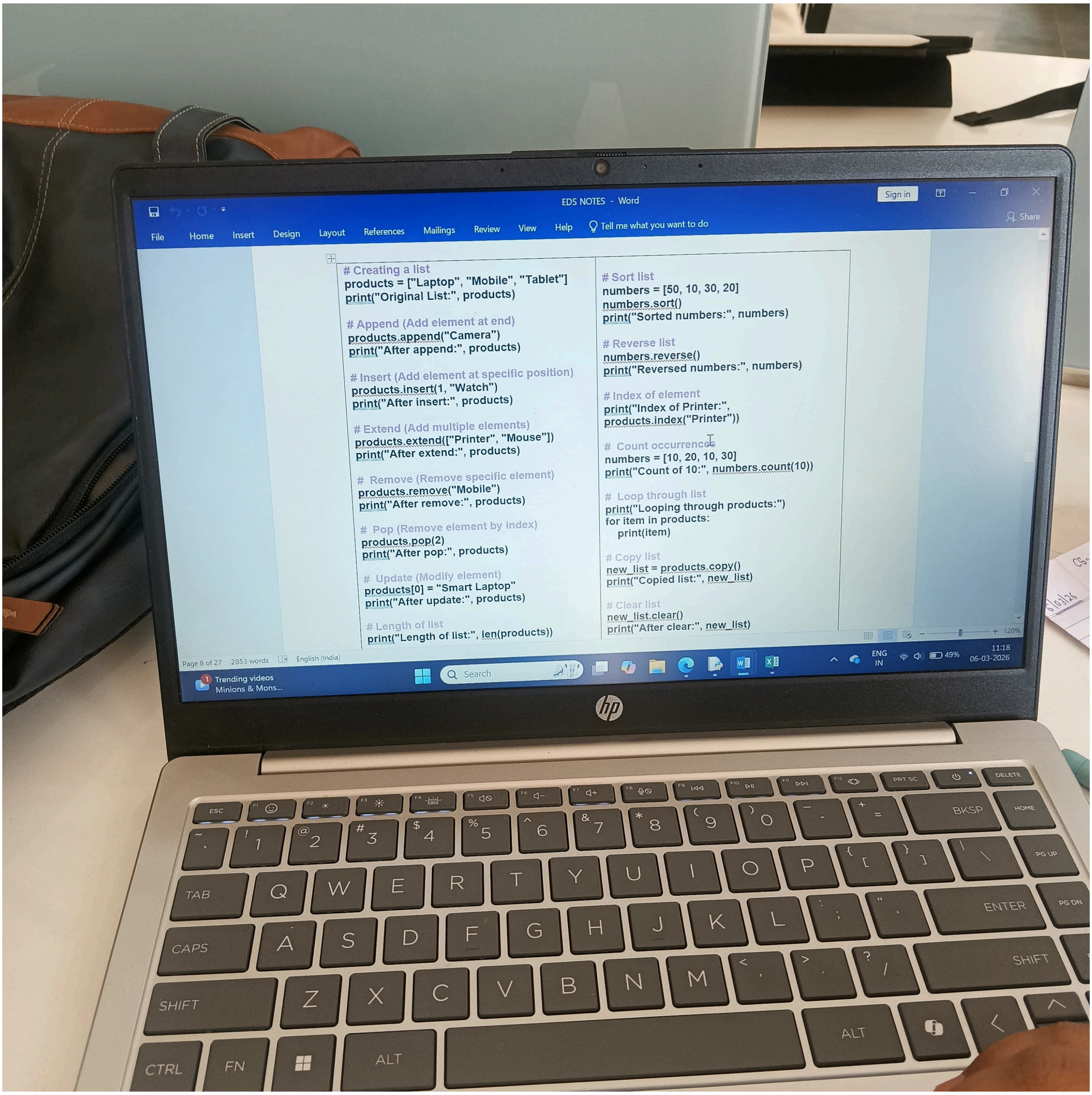


Dictionary Operations and Player Height Analysis

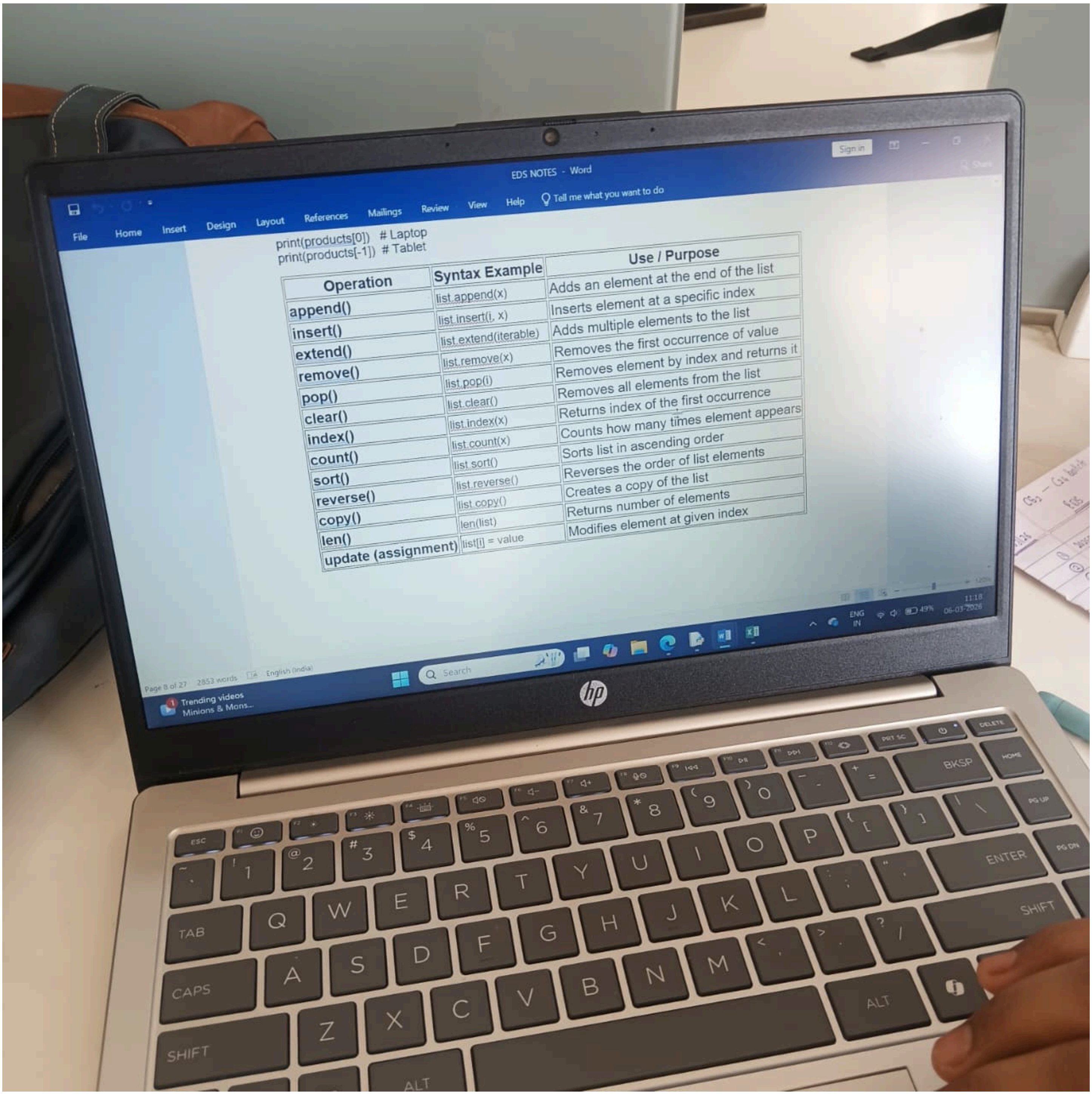
```python
team={"Virat":175, "Aman":180, "Amir":177, "Akash":182, "King":160}
c=max(team, key=team.get)
print(team)
print(c)
print(team[c])
```

```
IDLE Shell 3.11.9
File Edit Shell Debug Options Window Help
Python 3.11.9 (tags/v3.11.9:de54cf5, Apr  2 2024, 10:12:12) [MSC v.1938 64 bit (
AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: D:\new\06-03-2026\dictionary.py
{'Virat': 175, 'Aman': 180, 'Amir': 177, 'Akash': 182, 'King': 160}
Akash
182
>>>
```

Python List Methods Notes



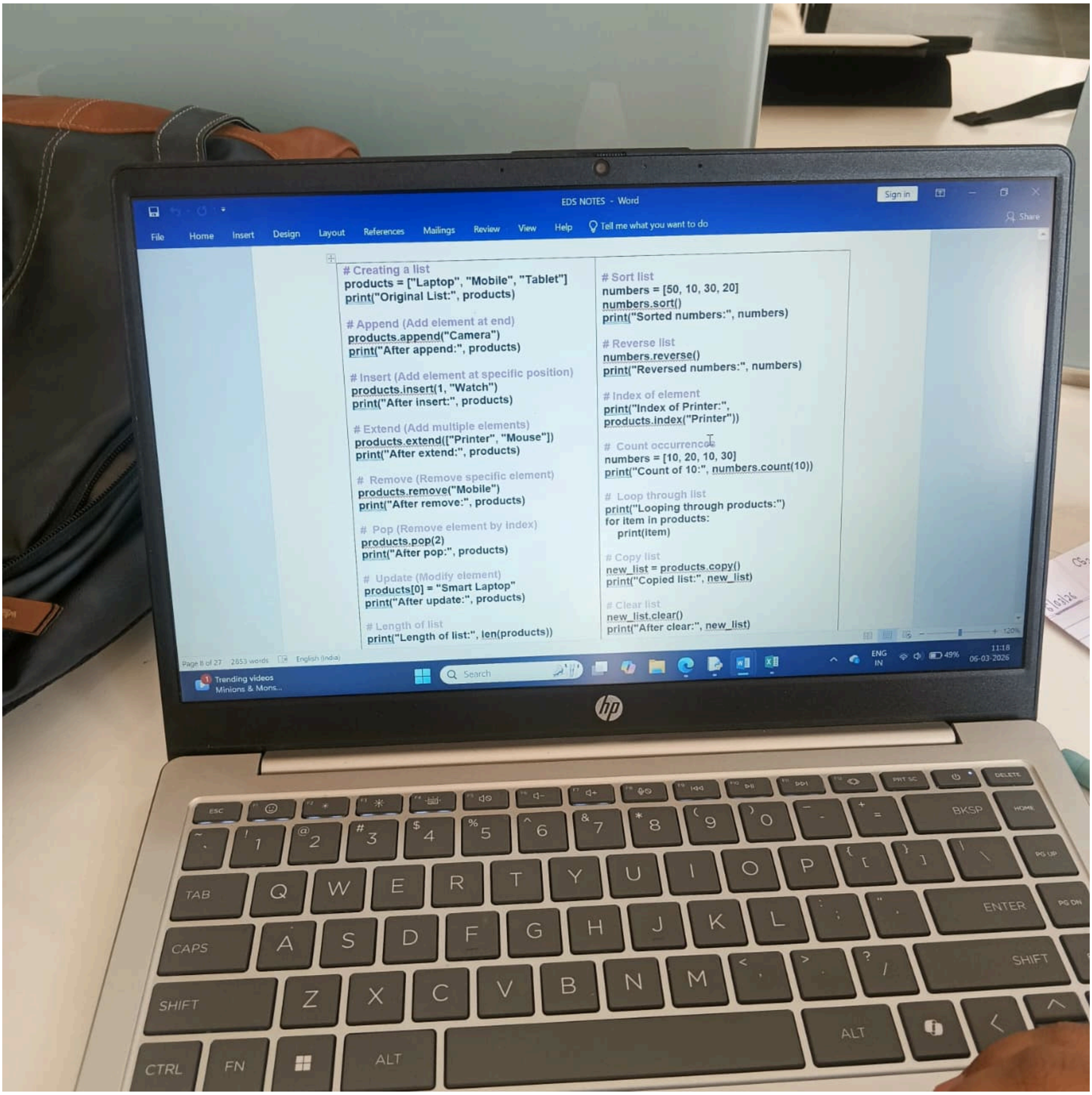
```python
# Creating a list
products = ["Laptop", "Mobile", "Tablet"]
print("Original List:", products)

# Append (Add element at end)
products.append("Camera")
print("After append:", products)

# Insert (Add element at specific position)
products.insert(1, "Watch")
print("After insert:", products)

# Extend (Add multiple elements)
products.extend(["Printer", "Mouse"])
print("After extend:", products)

# Remove (Remove specific element)
products.remove("Mobile")
print("After remove:", products)

# Pop (Remove element by index)
products.pop(2)
print("After pop:", products)

# Update (Modify element)
products[0] = "Smart Laptop"
print("After update:", products)

# Length of list
print("Length of list:", len(products))

# Sort list
numbers = [50, 10, 30, 20]
numbers.sort()
print("Sorted numbers:", numbers)

# Reverse list
numbers.reverse()
print("Reversed numbers:", numbers)

# Index of element
print("Index of Printer:",
products.index("Printer"))

# Count occurrences
numbers = [10, 20, 10, 30]
print("Count of 10:", numbers.count(10))

# Loop through list
print("Looping through products:")
for item in products:
    print(item)

# Copy list
new_list = products.copy()
print("Copied list:", new_list)

# Clear list
new_list.clear()
print("After clear:", new_list)
```



print(products[0]) # Laptop
print(products[-1]) # Tablet

Operation	Syntax Example	Use / Purpose
append()	list.append(x)	Adds an element at the end of the list
insert()	list.insert(i, x)	Inserts element at a specific index
extend()	list.extend(iterable)	Adds multiple elements to the list
remove()	list.remove(x)	Removes the first occurrence of value
pop()	list.pop(i)	Removes element by index and returns it
clear()	list.clear()	Removes all elements from the list
index()	list.index(x)	Returns index of the first occurrence
count()	list.count(x)	Counts how many times element appears
sort()	list.sort()	Sorts list in ascending order
reverse()	list.reverse()	Reverses the order of list elements
copy()	list.copy()	Creates a copy of the list
len()	len(list)	Returns number of elements
update (assignment)	list[i] = value	Modifies element at given index

List Operations Syntax and Examples Table



```
# Creating a list
products = ["Laptop", "Mobile", "Tablet"]
print("Original List:", products)

# Append (Add element at end)
products.append("Camera")
print("After append:", products)

# Insert (Add element at specific position)
products.insert(1, "Watch")
print("After insert:", products)

# Extend (Add multiple elements)
products.extend(["Printer", "Mouse"])
print("After extend:", products)

# Remove (Remove specific element)
products.remove("Mobile")
print("After remove:", products)

# Pop (Remove element by index)
products.pop(2)
print("After pop:", products)

# Update (Modify element)
products[0] = "Smart Laptop"
print("After update:", products)

# Length of list
print("Length of list:", len(products))

# Sort list
numbers = [50, 10, 30, 20]
numbers.sort()
print("Sorted numbers:", numbers)

# Reverse list
numbers.reverse()
print("Reversed numbers:", numbers)

# Index of element
print("Index of Printer:",
products.index("Printer"))

# Count occurrences
numbers = [10, 20, 10, 30]
print("Count of 10:", numbers.count(10))

# Loop through list
print("Looping through products:")
for item in products:
    print(item)

# Copy list
new_list = products.copy()
print("Copied list:", new_list)

# Clear list
new_list.clear()
print("After clear:", new_list)
```